

Pilarius: Multiplexing GPU Memory through Cyclic KV Restoration for Interactive Model Serving

Anonymous Author(s)

Abstract

Serving full-duplex interaction models requires maintaining ongoing sessions in which input and output can overlap. In periodic service, each update processes new input and generates output as needed, while historical key-value (KV) state persists between updates. As contexts grow, keeping this state on the GPU can exhaust memory while the combined work of admitted sessions still leaves time within each period. Offloading idle KV frees capacity, but restoring it only when the next update arrives can delay computation and cause deadline misses. The periodic schedule makes the next update time known in advance, allowing restoration to be planned before demand. We present PILARIUS, a serving system for multiplexing GPU memory across periodic interaction sessions. Through cyclic KV restoration, PILARIUS reuses space released by idle sessions to prepare other sessions for their next updates. It coordinates these restorations with session execution, accounting for both shared transfer capacity and GPU memory availability.

Placeholder. Measured session capacity and deadline-miss rates under matched service objectives and generation work, with workload and observation duration.

Keywords: KV cache, GPU memory management, model serving, full-duplex interaction, real-time inference, scheduling

1 Introduction

Full-duplex interaction is an emerging direction in human-AI communication, with recent models such as OpenAI’s GPT-Live and ByteDance’s Seeduplex [2, 9]. By listening while speaking and adapting to interruptions, these models let users clarify or redirect a conversation without waiting for a response to finish. At the serving layer, we treat each ongoing interaction as an *interaction session*.

We study interaction sessions that advance through periodic updates, each processing new input and generating output as needed. For example, Moshi operates on 80 ms audio frames, while Thinking Machines Lab’s TML-Interaction-Small organizes input and output into 200 ms *time-aligned micro-turns* [4, 11]. The session’s history persists across these updates.

To reuse history across updates, the server caches attention keys and values (KV), whose footprint grows with the

retained context. Using published parameters for MiniCPM-o 4.5, we estimate a BF16 KV footprint of about 1.2 GiB in the language backbone for a ten-minute audio conversation [3, 10]. This estimate assumes full retention of continuous input and output streams, contributing ten audio positions and four generated text tokens per second, respectively. It excludes prompt and control tokens and state in other modules.

Keeping all retained KV on the GPU, which we call *full residency*, consumes memory even during intervals when the session does not access it (Figure 1). As histories grow across concurrent sessions, this state can exhaust GPU KV capacity before their combined computation uses up the time available in each period. We target this regime, where reusing the space occupied by idle KV could allow the server to accommodate more sessions.

Offloading idle KV to host memory makes its GPU space available to other sessions. InferCept, for example, chooses among retaining, swapping, and recomputing KV for requests paused by external interactions [1]. In periodic service, the evicted state is needed again for the next update. We use the start of the next period as each update’s soft deadline. If restoration begins only when an update is released, computation that needs the evicted state must wait for the transfer, potentially causing a deadline miss (Figure 2(a)).

Periodicity provides an opportunity to avoid this delay: the next update’s scheduled release time is known in advance. We call the start of each period a *tick* and use it as the target for completing restoration. By keeping idle KV in host memory and restoring it before the next tick, the server can reuse the same GPU space across sessions. We refer to this reuse as *memory multiplexing*.

Scheduling restoration requires both transfer time and GPU space. Restorations compete for shared transfer bandwidth, and the full GPU destination for each restoration must be allocated before copying begins. Restoring too early occupies space that other sessions could use; starting too late risks missing the next tick. Larger evictions free more space but require more data to be transferred back. The server must therefore coordinate how much KV to evict from each session, when to restore it, and when to schedule each session’s computation.

We present PILARIUS, a serving system that multiplexes GPU memory across periodic interaction sessions through *cyclic KV restoration*. PILARIUS assigns each session a *phase*, or release offset within the period, to spread execution and restoration demand over time. After a session completes an

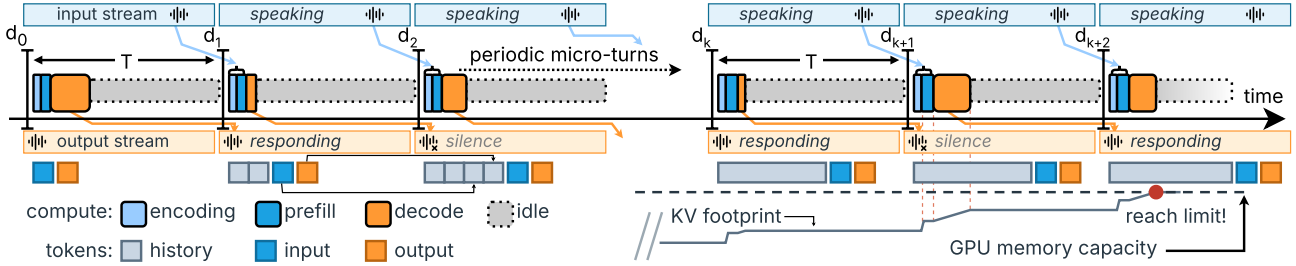


Figure 1. Historical KV remains allocated between updates under full residency. In this schematic example, each update encodes and prefills new input and may decode output for playback. Retained input and output extend the history, increasing the KV footprint toward the GPU capacity limit. The timeline and memory growth are illustrative.

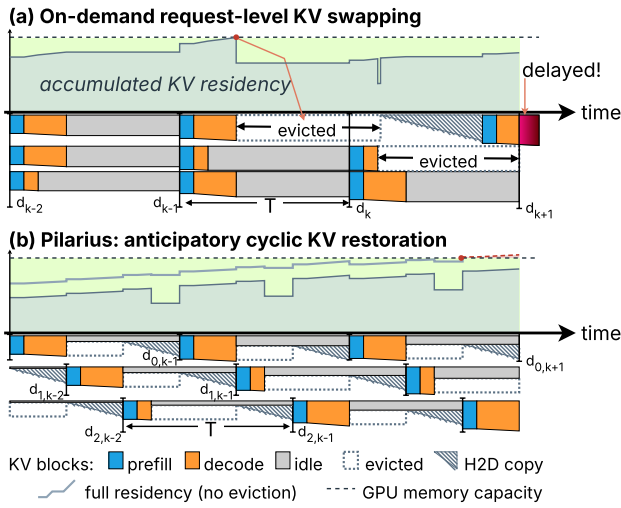


Figure 2. Restoration timing affects both computation and GPU memory use. (a) On-demand restoration delays dependent computation until the required KV is available. (b) Staggering sessions allows space released by one session to accommodate another’s restoration before its next tick. The upper curves show total GPU memory allocated to KV, including the full destination of each restoration as soon as it is allocated. Hatching denotes host-to-device (H2D) copies, dotted outlines denote evicted state, and gray denotes idle resident KV. The pale curve in (b) shows the allocation under full residency. Both panels are schematic.

update, PILARIUS evicts part of its idle KV and schedules restoration to complete before the next tick. The plan coordinates phases, eviction amounts, and restoration times so that space released by one session can accommodate another’s restoration (Figure 2(b)). The goal is to serve more concurrent sessions while preserving their retained histories and meeting the same soft deadline targets.

As session histories grow, a previously feasible schedule may no longer satisfy memory, transfer, or compute constraints. At admission, PILARIUS therefore plans over a finite lookahead interval, accounting for expected KV growth, computation, transfers, and GPU allocation. It reuses this plan across updates, with infrequent revisions to account for history growth and changes in the set of admitted sessions. These checks cover only the lookahead interval, so the server must reassess resource needs as sessions continue.

Placeholder. Final contributions and measured results: identify the configurations where KV capacity limits concurrency while compute time remains; establish the benefit attributable to coordinating phases, eviction amounts, and restoration timing; report concurrency and deadline-miss rates against full residency and on-demand restoration under matched service objectives.

2 Background and Motivation

2.1 Periodic Work and Playback Pacing

An interaction session repeatedly processes new input while carrying history across updates. With period T , each update is released at a tick and has a soft deadline at the next tick. Service latency includes both waiting and execution; delaying submission does not move the deadline. Once an update stops accessing its historical KV, that state can remain idle until the next update even while its output is being played. This interval creates an opportunity to reclaim GPU space while preserving the session’s history.

Speech illustrates why an update can finish well before its output has been consumed. In models trained for time-aligned interaction, generation follows playback progress: each update typically produces content for the corresponding playback interval. For example, MiniCPM-o 4.5 trains on text and speech assigned to temporal chunks and uses Time-Aligned Interleaving to adjust generation to accumulated playback progress [3]. A long response therefore unfolds across updates, with new input incorporated along the way. Playback determines the useful pace of generation, while the hardware can compute that content much faster.

Table 1. Cross-hardware analysis of the KV-capacity boundary (framework; values pending). G_{KV} excludes weights, other model state, and runtime allocations; L_{mem} is defined in Equation 1. The predicted cycle cost covers all N sessions, with ranges reflecting sensitivity to execution efficiency and workload variation. Evidence types will distinguish measurements from calibrated projections. Dashes denote unavailable values, and all bottleneck assessments remain pending.

GPU configuration	Estimated KV pool G_{KV} (GiB)	Capacity boundary L_{mem} (positions/session)	Predicted cycle cost $\widehat{C}_N(L_{\text{mem}})/T$ (range)	Evidence type	Assessment
A100 40GB PCIe	—	—	—	Pending	Pending
L40S 48GB	—	—	—	Pending	Pending
RTX 5090 32GB	—	—	—	Pending	Pending
A100 80GB PCIe	—	—	—	Pending	Pending
H100 80GB PCIe	—	—	—	Pending	Pending
RTX 4090 24GB	—	—	—	Pending	Pending

For a concrete scale, MiniCPM-o 4.5 uses one-second updates, with ten incoming audio positions and about three to four generated text tokens at normal speaking speed [3]. These counts exclude prompt and control positions. Its public llama.cpp-omni implementation reports an audio prefill stage of about 21 ms and a backbone decoding cost of about 38 ms/token on an RTX 4090 in F16 [7]. Combining these external figures gives 114–152 ms of backbone decoding per second of speech, or about 6.6–8.8 times the playback rate, and 135–173 ms including the reported prefill stage. This is a scale estimate, not a complete cycle measurement: speech synthesis and runtime overhead also require time, and the reported stage timings do not specify batch size or retained context length. Those costs must be measured at the intended concurrency and history length.

2.2 Why KV Capacity Can Bind First

Playback pacing keeps typical generation per update small, while the retained history accumulates over successive updates. The session can therefore add only a small amount of work and state each period while eventually retaining a large KV cache. Keeping that cache on the GPU throughout the cycle consumes capacity even while the session is idle. Training does not impose a strict token cap, and attention becomes more expensive as history grows; the resulting margin therefore depends on both the workload and its execution cost.

The relevant comparison is between the memory and computation required by all admitted sessions. Consider N sessions with a common period, L retained positions each, and κ KV bytes per position, with no KV sharing. Let G_{KV} be the GPU capacity available for KV after other allocations, and let $C_N(L)$ be the time required for the complete set of periodic work under the chosen execution policy. It includes input processing, generation, and runtime overhead, with batching accounted for. Ignoring allocation rounding, memory binds

first when

$$L_{\text{mem}} = \frac{G_{KV}}{N\kappa}, \quad C_N(L_{\text{mem}}) < T. \quad (1)$$

The remaining budget is $T - C_N(L_{\text{mem}})$. Equation 1 states the operating condition that admission must establish: the complete workload still has time available when full residency fills the KV pool. This is the margin that can absorb the computation and interference costs of memory multiplexing. The resulting schedule must also satisfy each session’s deadline.

Placeholder. Measure complete periodic work at the full-residency capacity limit, including the distribution of per-update generation, context lengths, batching, and phase-dependent execution costs.

Table 1 provides a framework for examining this condition across hardware configurations with a common model, precision, period, session count, and workload. It pairs the estimated capacity boundary with the predicted fraction of the cycle needed to execute the workload at that boundary. The execution estimates require calibration and independent validation; peak hardware specifications alone do not establish that the workload finishes within the period.

2.3 Bandwidth Limits on Memory Reuse

Offloading replaces continuous GPU residency with repeated restoration. If session i evicts e_i bytes per cycle and B_{eff} is the effective H2D bandwidth available to restoration, then the total restored state obeys

$$E = \sum_i e_i \leq B_{\text{eff}}T. \quad (2)$$

The utilization of this transfer budget is $E/(B_{\text{eff}}T)$; continuous use reaches the ceiling $B_{\text{eff}}T$. The bandwidth must account for competing traffic, including incremental host backups. Each copy must also fit between safe eviction and the next tick and obtain its GPU destination before it starts.

Why full eviction can fail. Evicting every session’s entire history makes transfer demand grow with total retained

state. Once that demand exceeds Equation 2, all histories cannot be restored every period even with continuous transfers. A single large restoration can also exceed its available window or require more destination space than is free. For illustration, assume eight histories of 4.5 GiB, a one-second period, and 24 GiB/s of effective H2D bandwidth. Full restoration needs 36 GiB per cycle, or 1.5 seconds of copying; evicting 2 GiB per session reduces this to 16 GiB and about 0.67 seconds. These are assumed resource values, not measured capacity results; the partial plan still needs feasible copy windows and GPU allocations. Partial eviction allows the server to retain the state that cannot be restored in time.

From transferred bytes to usable capacity. A byte saves GPU space only from its eviction until its restoration destination is allocated. For a steady cycle with fixed history sizes and no shared KV, let w_i be the average time that session i 's e_i evicted bytes spend without a GPU allocation. Let ΔG_{peak} be the reduction in peak allocation relative to full residency and $\overline{\Delta G}$ the average reduction over the cycle. Then

$$\Delta G_{\text{peak}} \leq \overline{\Delta G} = \frac{1}{T} \sum_i e_i w_i \leq B_{\text{eff}} T. \quad (3)$$

Thus, $B_{\text{eff}} T$ bounds the state that can be reused each cycle. The capacity gain depends on the duration and overlap of the eviction intervals. Earlier destination allocation shortens these intervals. Staggering sessions helps one session's eviction supply space for another's restoration, but both their memory overlap and execution cost must fit the schedule.

3 System Design

3.1 Architecture and Execution Cycle

PILARIUS coordinates session phases, eviction budgets, and restoration timing to reuse GPU space between updates. It operates above an inference backend, which remains responsible for execution scheduling and model computation. Batch formation, when used, remains a backend policy; the architecture does not require continuous batching. A residency planner constructs the plan at admission and revises it as resource demands change. A session manager follows the plan, using ticks, execution events, and KV readiness to submit work to the backend. A KV memory manager tracks allocation, valid copies, and references, while a KV transfer engine executes asynchronous copies. The session manager issues eviction and restoration requests to the memory manager, which submits transfer requests with valid sources and allocated destinations. A transfer completion reports a finished copy; KV readiness is published only after all required ranges are valid for the corresponding session. These components separate planning decisions from the safe execution of each cycle.

Consider one session in Figure 1. At its tick, historical KV is ready on the GPU; the backend processes new input and generates the next output. Newly retained KV is backed up to

host memory. After the selected ranges are no longer in use and their host copies are valid, PILARIUS releases their GPU storage. Other sessions can use this space until the manager allocates a destination and restores the evicted state before the next tick. Figure 2(b) shows this cycle across staggered sessions.

3.2 Session Groups and Phase Assignment

PILARIUS divides a common period T into S phase slots, with release offsets $\phi_j = jT/S$ for $j = 0, \dots, S - 1$. Sessions assigned to the same slot form a session group and become eligible for execution at the same phase. Each occupied slot can contain multiple sessions; the design does not require one slot per session. Sessions retain separate histories and eviction budgets, and the backend forms execution batches from the eligible work. A session group is therefore a scheduling unit, with no requirement that all members execute as one fixed batch or transfer atomically.

Trading batching efficiency for memory reuse. Aligning every session to the same phase makes more work available for batching, but also concentrates demand for resident KV and restoration. Staggering groups distributes that demand and lets their eviction intervals supply space for one another. It can reduce batch sizes and increase total execution cost through additional model iterations and weight accesses. Grouping multiple sessions at each phase preserves opportunities for batching while avoiding a single synchronized burst. For roughly balanced groups, increasing S reduces the number of sessions available per group from N toward N/S . The planner must choose this tradeoff using the backend's measured execution costs.

The goal is to preserve the service target while reducing peak memory use. Let C_{aligned} and C_{grouped} denote the complete periodic execution costs for the two release patterns at the same history length and workload. The extra cost $C_{\text{grouped}} - C_{\text{aligned}}$, together with transfer interference and scheduling delays, must fit within the original margin $T - C_{\text{aligned}}$. The short generation bursts described in Section 2.1 explain why a moderate batching penalty can leave useful slack. Admission checks this margin and each session's completion time under the proposed grouping. Initial phase alignment and any resulting input delay count toward the service cost.

3.3 Joint Residency Planning

The planner chooses phase assignments, per-session eviction amounts, and restoration times over a finite lookahead horizon H . It uses the configured period, retained state, expected growth, execution costs under the proposed groups, host capacity, and effective transfer service. A free phase slot is insufficient for admission: the proposed sessions must fit the compute budget, individual transfer windows, and instantaneous GPU capacity. Equation 3 shows why transfer

utilization alone does not establish a capacity benefit; the planner must reduce peak allocation.

Eviction budgets. An eviction budget e_i limits the amount of safely recoverable KV released from session i . It need not cover the entire history. As Section 2.3 shows, full eviction eventually exceeds the transfer budget as retained state grows. Partial eviction keeps the excess resident and gives the planner control over each restoration's duration and destination size. A useful budget must free enough memory to matter while leaving a feasible return path.

Restoration timing and space. Let $r_{i,k+1}$ be the next tick and u_i the planned restoration start. Given the effective bandwidth assigned to the copy, the estimate $u_i + e_i/\widehat{B}_i + \delta_i \leq r_{i,k+1}$ must include queueing, completion, and safety allowances in δ_i . The individual estimates must agree with a feasible shared transfer schedule and Equation 2. At every point, total physical GPU allocation must remain within G_{KV} . This accounting includes the entire destination of each copy from the time it is allocated, with shared blocks counted once. Starting early can shorten the interval of reclaimed space even when it makes the transfer easier to finish.

The planner therefore checks when earlier groups release space as well as when later groups need it. It must account for expected history growth throughout H ; a plan that fits the current state may cease to fit before the horizon ends.

Placeholder. Joint planning algorithm: selection of group count and horizon, session assignment, safe eviction ranges, resource estimates and margins, and the time-and-space feasibility check.

3.4 Safe Eviction and Scheduled Restoration

Backing and releasing state. The memory manager incrementally creates host copies of newly retained KV; valid copies of unchanged history can be reused across periods. Backing up state does not release its GPU allocation. Eviction requires both a complete host copy and the absence of conflicting execution or transfer references. For a shared physical block, all references must permit reclamation. These conditions preserve the application's selected history while changing where its KV resides.

Allocating and copying. The session manager requests restoration at the planned time. The memory manager allocates the full destination before submitting a copy to the transfer engine. Space reclaimed from another group's idle state can satisfy this allocation. An ongoing copy already occupies GPU capacity, but its contents become usable only after completion is published.

Placeholder. Choose and justify the restoration allocation boundary: whole-group reservation or per-session allocation under the group schedule; specify readiness and recovery when a later allocation cannot proceed.

Resuming computation. The backend may access restored KV only after all required ranges are valid for the correct session and logical positions. Concurrent demand, prefetching, and eviction must share a consistent view of allocation and content validity. If space or transfer service is unavailable at the planned time, the manager must reassess completion before allowing dependent computation. Demand restoration can recover missing state, but its delay remains part of service latency and can cause a deadline miss. No timeout permits reading unfinished KV or reusing a referenced block.

3.5 Plan Reuse and Revision

The planner reuses its plan across updates when state growth remains sufficiently predictable over the horizon. Per-cycle checks enforce current space, reference, and readiness conditions without reselecting every phase and budget. Accumulated history growth requires infrequent revisions; session arrivals and departures can require larger adjustments. Keeping some KV resident limits transfer traffic, but that resident portion can still grow and eventually exhaust the available capacity.

Revisions must account for allocations and transfers already in progress. A new plan cannot release a referenced block or turn an unfinished copy into valid state. Feasibility within H therefore supports only that planning horizon, with continued service requiring reassessment.

Placeholder. Revision triggers, horizon extension, transition between plans with in-flight work, and service policy when no feasible revision exists.

3.6 Backend Integration

The backend interface must expose request submission, execution completion, KV block identity, allocation, references, and copy readiness. PILARIUS coordinates these operations with the backend's existing block allocator so that memory accounting and validity have one consistent authority. The backend controls execution scheduling and any batching policy; session groups determine which work becomes eligible together.

Placeholder. Verified integration: identify the backend and implemented planning and recovery paths, synchronization points, and remaining gaps relative to this design.

4 Evaluation

The evaluation asks whether coordinating phases and KV restoration increases session capacity under a fixed service objective, and which resource limits that increase. It separates the main capacity comparison from the measurements needed to explain it: memory pressure, restoration delay, mechanism ablations, and behavior as histories grow.

Placeholder. Experimental results are pending. Specify the evaluated models, workloads, platforms, software versions, and repetition protocol before reporting measurements.

4.1 Methodology

Workload and comparison controls. A residency comparison holds the model, precision, input content, retained history, and generation work fixed. The controls also match batching limits, scheduling policies, and resource budgets except for the variable under study. Retention policies are workload parameters; each residency comparison preserves the selected policy and its resulting history. Workload parameters include the period, session duration, context-length range, and input and output configuration.

Compared systems. Full residency establishes capacity and latency without recovery. On-demand host restoration exposes the cost of putting recovery on the execution path. Recomputation provides a second way to recover missing state when the required historical inputs are available. Comparisons with PILARIUS use the same generation workload and include the phase and restoration ablations described below.

Placeholder. Named external baselines, exact configurations, host-copy policies, and any adaptations needed for the periodic workload.

Service capacity. The main metric is the largest concurrent session population that satisfies the specified deadline-miss and failure criteria over the observation interval. Offered, admitted, and completed work are reported separately, so refusing more load cannot appear as a latency improvement. Deadline violations use $c(i, k) > r(i, k + 1)$, and latency distributions use $c(i, k) - r(i, k)$. The analysis also records initial phase-alignment delay and accumulated backlog. Progress measurements distinguish legitimate silence from missing generation work, which a timely empty response can conceal.

Placeholder. Allowed miss rate, session-failure criteria, observation interval, load sweep, repetitions, and uncertainty reporting.

Resource measurements. GPU memory measurements include all KV allocations, particularly destinations whose copies have not completed. Valid content, evicted bytes, and host-copy storage are recorded separately. Transfer measurements distinguish waiting for space, queueing for the copy stream, copying, and completion handling. Compute measurements cover the work required by each micro-turn and its interference with transfers. These measurements connect a change in session capacity to the resource it consumes.

4.2 Capacity and Service Latency

The main comparison varies concurrency and session duration to locate the service limit of each system. Latency and deadline violations at equal offered load show whether recovery introduces a cost before capacity is exhausted. At each system's capacity limit, the allocation and execution traces identify whether memory, transfers, computation, or growing backlog prevents further admission.

Placeholder. Supported concurrency and service-latency distributions for all systems, with completed work and failure outcomes.

The full-residency limit. The motivating measurement varies retained history and concurrency together. A memory-limited operating point requires evidence that KV allocation prevents additional service while the corresponding computation still fits within the period. Session-level idle intervals alone do not establish spare compute capacity across the workload. Near the memory boundary, the measurements report the complete workload's execution time per period and the remaining fraction of the period, rather than inferring slack from a device-utilization counter. Calibration and validation use separate configurations to test Equation 1, including regions where it predicts no slack. An additional-work experiment tests whether the observed budget can accommodate more session work while preserving the service objective.

Placeholder. Joint allocation and execution traces at the full-residency limit, predicted versus measured slack, and additional work completed within the same deadline.

4.3 Mechanism Ablations

The ablations isolate phase assignment from advance restoration. The first comparison holds the restoration policy fixed and changes aligned releases to uniformly staggered releases. The second keeps phases fixed and compares three recovery policies: on-demand restoration, prefetching after submission, and restoration planned before the next tick. Their combination tests whether spreading demand improves the memory and transfer schedule beyond either action alone.

A further comparison holds the phase schedule fixed while varying eviction budgets and restoration lead time. It tests the coupling in Section 2.3: too little eviction leaves memory occupied, whereas excessive eviction or early allocation can make the schedule infeasible. Host-copy policy, retained history, and transfer implementation remain fixed within each comparison.

Placeholder. Ablation results: session capacity, peak allocation, transfer utilization, deadline violations, and the selected per-session budgets.

Restoration on the critical path. For each micro-turn, the analysis measures how much restoration finishes before its tick and how much delays computation. Allocation waiting, copy-stream queueing, transfer time, and post-transfer scheduling are distinguished. This decomposition tests whether the design removes recovery delay or moves it to an earlier queue.

Placeholder. Restoration timelines and latency decomposition at matched load, including waiting before submission.

4.4 History Growth and Plan Revision

Admission predictions are compared with execution over configurations and traces not used for calibration. The comparison covers per-period KV growth, transfer duration, computation time, and peak allocation. Prediction error is reported against the horizon and scheduling margins used to make the admission decision.

Continuous runs span multiple planning horizons to test service beyond the initial admission decision. One experiment keeps the session population fixed while histories grow; another introduces departures and new admissions. Together they distinguish gradual correction from population changes. The relevant outcomes include revision frequency and cost, service around each revision, and the treatment of sessions when a new feasible plan cannot be found.

Placeholder. Prediction errors, revision behavior, and continuous service across horizons; a successful initial horizon alone does not establish sustained capacity.

4.5 Correctness and Operating Limits

Correctness comparisons preserve the same history and model inputs as the reference execution. They exercise state round trips and overlapping demand, restoration, eviction, and plan changes. For copying, the correctness criterion is preservation of KV content; any recomputation path additionally requires a numerical and output-equivalence criterion.

Placeholder. Correctness criteria and results, including delayed copies, allocation pressure, and concurrent operations.

Overheads. The sensitivity analysis varies the period, retained history, usable KV capacity, effective transfer bandwidth, and host capacity. These experiments identify where recovery consumes the available slack or releases too little memory to increase service capacity. The comparison includes scheduling and host-backup overhead, along with the effect of staggering on batching efficiency.

Placeholder. Measured overheads and limits, including late input and transfer contention.

5 Discussion and Limitations

Applications without phase control. An application may expose future ticks while requiring fixed release phases. Advance restoration can still use that timing information, but the manager cannot redistribute demand by assigning phases. The benefit then depends on the supplied schedule and available memory. The phase ablation in Section 4.3 separates these two sources of benefit.

Finite-horizon admission. A finite prediction horizon limits how far admission can anticipate history growth and interference. Smooth growth can permit infrequent revisions, but it does not guarantee that a feasible schedule exists indefinitely. If state keeps growing, memory, host capacity, or recovery time can eventually become limiting. The reported capacity is consequently tied to session duration, the prediction horizon, and the policy for handling infeasible revisions.

Future work: recomputation. PILARIUS currently plans recovery through H2D transfers. Separating planning from execution provides a way to incorporate KV recomputation when transfer bandwidth becomes limiting. The planner could add a decision variable for how much evicted KV to rebuild from retained inputs, jointly scheduling this work with H2D restoration. Execution would require a backend that supports replay and reports when reconstructed KV becomes valid.

Feasibility would depend on compute slack across all admitted sessions and space for reconstructed KV and temporary work. Both forms of recovery would need to finish by the next tick while leaving enough computation time for sessions to meet their soft deadlines. We leave this extension and its evaluation to future work.

Workload and platform dependence. A small retained history leaves little memory to reclaim, and a short interval between uses leaves little time to restore it. Effective transfer bandwidth, host-copy availability, and batching efficiency determine the useful eviction budget. Different periods or execution profiles can also produce unequal demands despite uniformly spaced phases. Uniform spacing spreads release times but does not optimize for these differences in resource demand.

Transfers across multiple devices. With multiple GPUs, restoration may share PCIe switches, host-memory channels, and other links. The effective bandwidth in Equation 2 must reflect contention on the paths used by the plan; adding nominal per-device bandwidths can overestimate the available service. Extending the planner requires tracking each constrained path together with the destination capacity on each device.

6 Related Work

Scheduling and KV allocation. Orca schedules generation at iteration granularity, allowing batch membership to change as requests arrive and finish [12]. PagedAttention maps logical KV blocks to noncontiguous physical storage, reducing fragmentation and enabling sharing in vLLM [6]. vLLM also supports swapping and recomputation when memory pressure requires preemption. These techniques improve how active work uses computation and memory. The residency problem considered here remains when the valid KV retained by periodic sessions exceeds GPU capacity. PILARIUS coordinates when that state occupies the GPU between uses; it still needs an execution engine and allocator that use the resulting capacity efficiently.

State recovery during interruptions. InferCept manages requests whose generation pauses for interactions with external tools or environments [1]. It estimates interruption durations, chooses among preserving, swapping, and recomputing KV, and chunks recovery work to reduce interference with other requests. This work establishes that idle-state decisions should consider both memory occupancy and recovery cost. Our setting adds a recurring restoration target and, where permitted, control over session phases. The design uses these properties to coordinate repeated recovery with the allocation of its destinations.

Combining loading and recomputation. Cake prepares cached prefixes by concurrently loading some KV chunks and recomputing others [5]. Its division follows causal attention costs: recomputation proceeds from earlier positions while loading proceeds from later positions. This use of computation to supplement loading motivates the future extension discussed in Section 5. Periodic restoration additionally requires fitting repeated preparation into a shared schedule as retained histories grow.

Latency-constrained serving. DistServe separates prefill and decoding onto different GPUs and allocates resources to satisfy time-to-first-token and time-per-output-token objectives [13]. Its design addresses interference between execution stages with different latency requirements. PILARIUS instead coordinates retained state across successive periods of an ongoing session, including time when that session performs no computation. Both designs require evaluating useful service under a latency objective rather than reporting throughput alone.

Periodic interaction and bounded history. Metronome studies memory exhaustion in periodic interaction serving and combines an in-engine KV window with deadline-aware admission [8]. Its window bounds state by retaining recent tokens and attention sinks. Its evaluation also uses phase-staggered input streams. PILARIUS manages the GPU residency of the application’s selected history through partial

eviction and advance restoration. Residency scheduling can apply to either full or bounded histories, trading host capacity and recurring recovery work for GPU capacity. A residency comparison holds the retention policy fixed so that its effect is measured separately from changes in retained history.

7 Conclusion

Periodic interaction exposes repeated opportunities to release a session’s idle KV and restore it before the next tick. Using those opportunities requires coordinating transfer time with the GPU space occupied by restoration destinations. PILARIUS combines uniform phase assignment with per-session eviction and restoration planning over a finite horizon, followed by infrequent revisions as histories and session populations change. The design seeks to increase service capacity while preserving retained history and meeting recurring soft deadlines.

Placeholder. Measured capacity, latency, and costs, with workload, duration, and resource conditions.

References

- [1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. <https://proceedings.mlr.press/v235/abhyankar24a.html>
- [2] ByteDance Seed. 2026. Introducing Seed Full-Duplex Speech LLM: Attentive Listening, Robust Interference Suppression, Enabling More Natural Interaction. <https://seed.bytedance.com/en/blog/introducing-seed-full-duplex-speech-llm-attentive-listening-robust-interference-suppression-enabling-more-natural-interaction>
- [3] Junbo Cui, Bokai Xu, Chongyi Wang, Tianyu Yu, Weiyue Sun, Yingjing Xu, Tianran Wang, Zhihui He, Wenshuo Ma, Tianchi Cai, Jiancheng Gui, Luoyuan Zhang, Xian Sun, Fuwei Huang, Moye Chen, Zhuo Lin, Hanyu Liu, Qingxin Gui, Qingzhe Han, Yuyang Wen, Huiping Liu, Rongkang Wang, Yaqi Zhang, Hongliang Wei, Chi Chen, You Li, Kechen Fang, Jie Zhou, Yuxuan Li, Guoyang Zeng, Chaojun Xiao, Yankai Lin, Xu Han, Maosong Sun, Zhiyuan Liu, and Yuan Yao. 2026. MiniCPM-o 4.5: Towards Real-Time Full-Duplex Omni-Modal Interaction. arXiv:2604.27393 [cs.CL] doi:10.48550/arXiv.2604.27393
- [4] Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. 2024. Moshi: A Speech-Text Foundation Model for Real-Time Dialogue. arXiv:2410.00037 [eess.AS] doi:10.48550/arXiv.2410.00037
- [5] Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Z. Morley Mao. 2025. Compute Or Load KV Cache? Why Not Both? arXiv:2410.03065 [cs.LG] doi:10.48550/arXiv.2410.03065
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. Association for Computing Machinery, 611–626. doi:10.1145/3600006.3613165
- [7] llama.cpp-omni Contributors. 2026. llama.cpp-omni: Architecture and Performance Benchmarks. <https://github.com/tc-mb/llama.cpp-omni/blob/3dfebb53caaa1ae9cdf5e71ec4db1db65effdfad/README.md>

- [8] Jiaying Meng and Bojie Li. 2026. Metronome: Bound the Cache, Keep the Beat for Real-Time Interaction Model Serving. arXiv:2607.02640 [cs.SD] doi:10.48550/arXiv.2607.02640
- [9] OpenAI. 2026. Getting Started with GPT-Live. OpenAI API Documentation. <https://developers.openai.com/api/docs/guides/live>
- [10] OpenBMB. 2026. MiniCPM-o-4_5 Model Configuration. https://huggingface.co/openbmb/MiniCPM-o-4_5/blob/503e754207c94da6bb26850b4469f367c9ea3582/config.json
- [11] Thinking Machines Lab. 2026. Interaction Models: A Scalable Approach to Human-AI Collaboration. *Thinking Machines Lab: Connectionism* (May 2026). doi:10.64434/tml.20260511
- [12] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 521–538. <https://www.usenix.org/conference/osdi22/presentation/yu>
- [13] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>